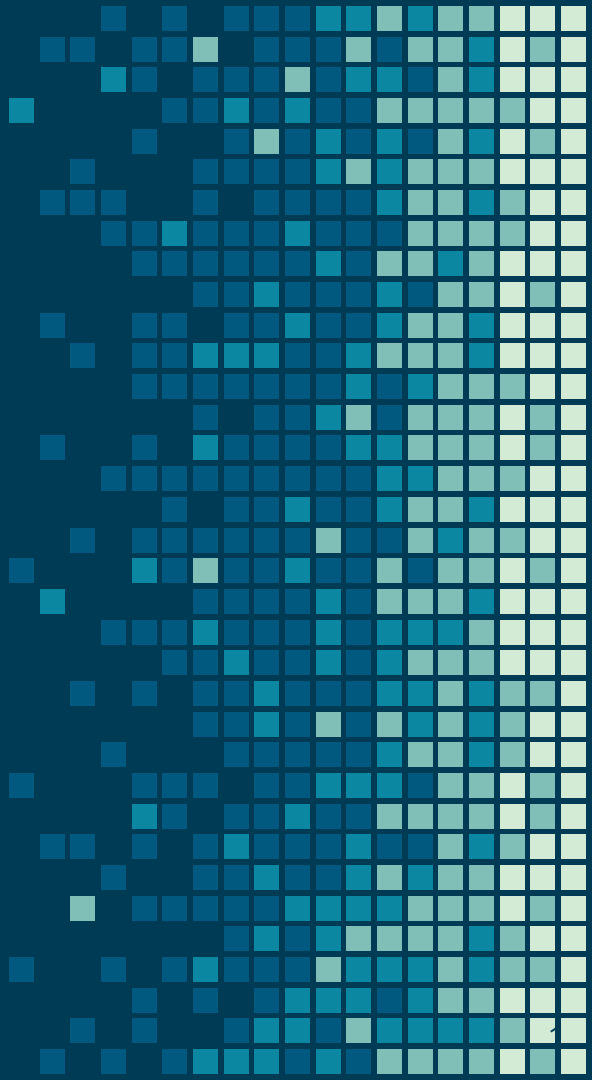
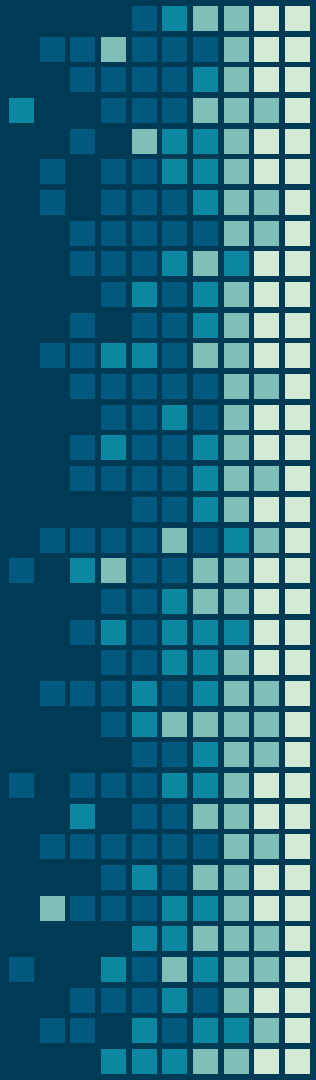
EMBEDDED CONTROL

Prof. Luca De Cicco
luca.decicco@poliba.it
https://c3lab.poliba.it/LDC



TIMERS



A (naive and wrong) control loop implementation

```
while(1) {
   read_outputs();/* read feedback from sensors*/
   compute_control_action(); /* the controller */
   actuate_control_action(); /* feed actuators */
   sleep(sampling_time); /* Implement a fixed
                            sampling time */
}
```

DISCLAIMER: this loop is broken and only works for very simple systems! DO NOT USE IT!!!

A better (not-so-good) alternative: periodic task

```
for each sampling_time:
    read_outputs();
    compute_control_action();
    actuate_control_action();
```

- Very close to the textbook implementation of a digital controller
- To implement this control loop in a firmware we need a time-base, i.e. a timer that "calls" the body of the control loop repeatedly at a fixed frequency
- A time-base is essentially a "configurable clock"

Timers

- Having the notion of time is fundamental for embedded control system design
- Scheduling precise delays is a key feature needed to implement digital controllers (recall Digital Control exam)
- Control algorithms are mainly difference equations that need to update a number of variables at a fixed time step
- The MCU core clocks are too fast, we need a way to slow them down and get notified when a specific time delay has passed
- Timers can be used to do much more than keeping track of time running!

Timers in MCUs

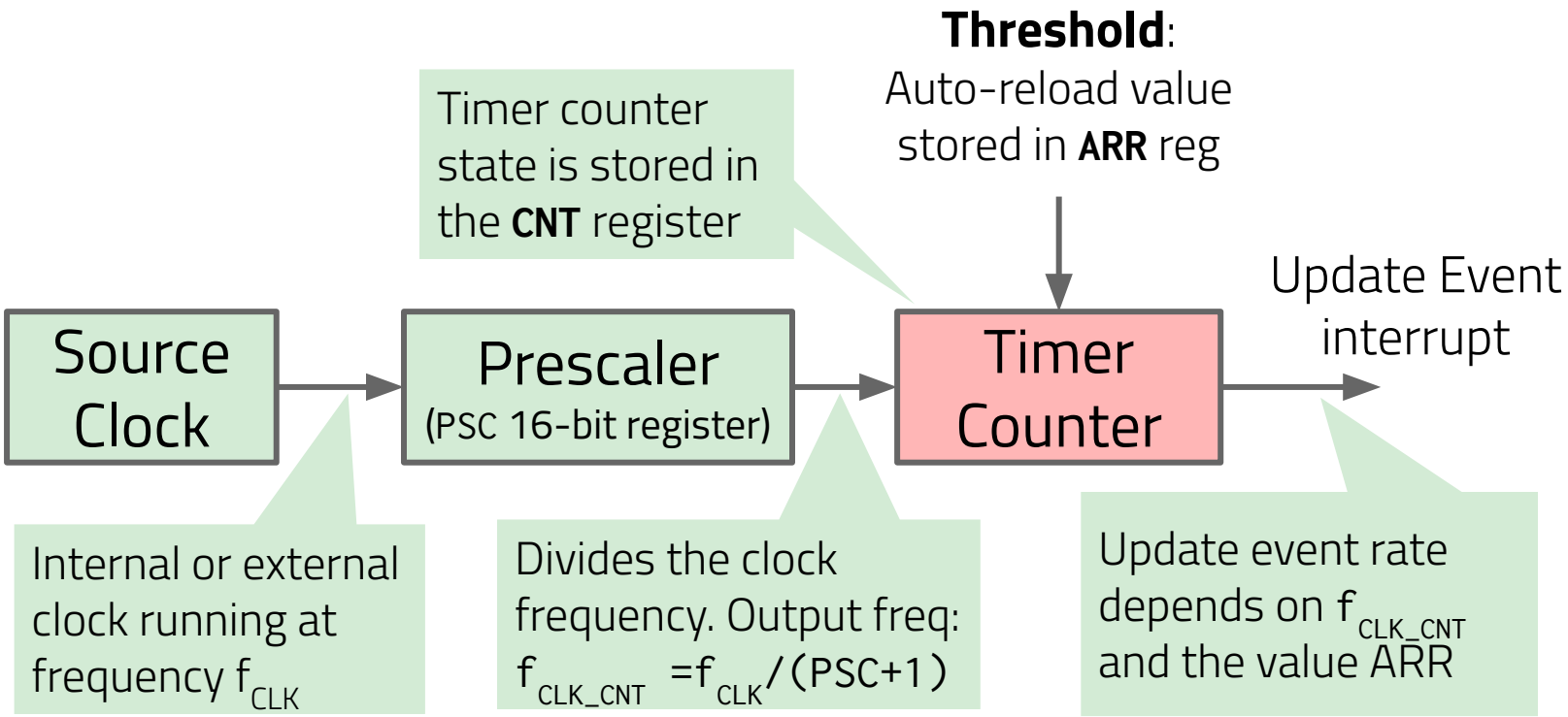
- All MCUs implement a number of specific peripherals for timing purposes
- In the STM32 MCUs different types of timers are available
- Timers are configurable pieces of hardware that assolve to different purposes
- Timers are quite complex, we will only cover the basics that are needed for our purposes (250+ pages in reference manuals)

Timers: definitions

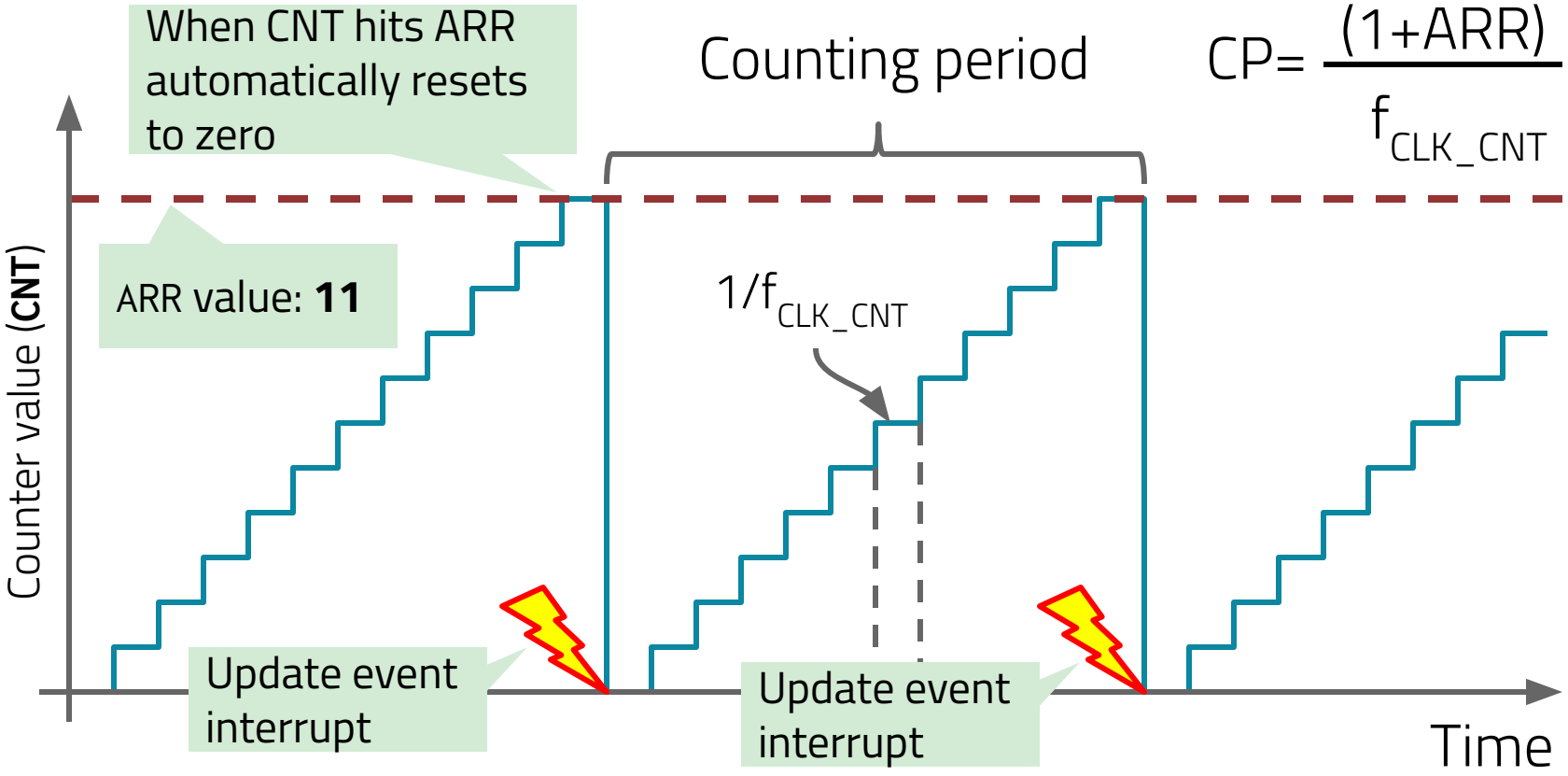
A **timer** is a free-running counter with a **counting frequency** that is a **fraction** of its **source clock**

- Depending on the timer type, the **clock source** can be:
  - the clock of the bus the timer is connected to
  - an external clock (through an MCU PIN)
  - another timer (slave mode)
- The **counting frequency** provided by the clock source can be divided by an integer amount through an hardware called **prescaler**

How to generate events at configurable rate



Example: a timer counting up



Timer resolution and counting modes

- **Timer resolution**: Timers can be 16-bit or 32-bit
- 16-bit timers can count up to 2^16, thus `ARR` and `CNT` can be maximum equal to 2^16
- Timers can be configured to generate interrupts when counter is reset
- Timers can have different counting modes:
  - **up-counting**: starts from 0, reaches ARR, then reset to 0
  - **down-counting**: starts from ARR, goes down to 0, then reset to the ARR
  - **center-aligned counting**: performs up-counting and down counting alternatively generating a triangle wave (doubles the period because goes up, then down!)

Example: generate an event every **500ms**

- Consider a timer connected to APB1 bus
- The clock for APB1 bus is f_{CLK}=90MHz (max for our micro)
- We need to tune the auto-reload register value (ARR) and the prescaler value (PSC) to get the desired period

$$\textbf{Period} = \frac{(1+ARR) \times (PSC+1)}{f_{CLK}}$$

- We could try to divide 90MHz with a 89999 to get $1/f_{CLK_CNT}$=1ms counting frequency , but PSC is 16-bit!
- We set **PSC=8999** => $1/f_{CLK_CNT}$=0.1ms, to get Period=500ms we need to multiply by 5000 => **ARR = 4999**

Trade-offs setting PSC and ARR

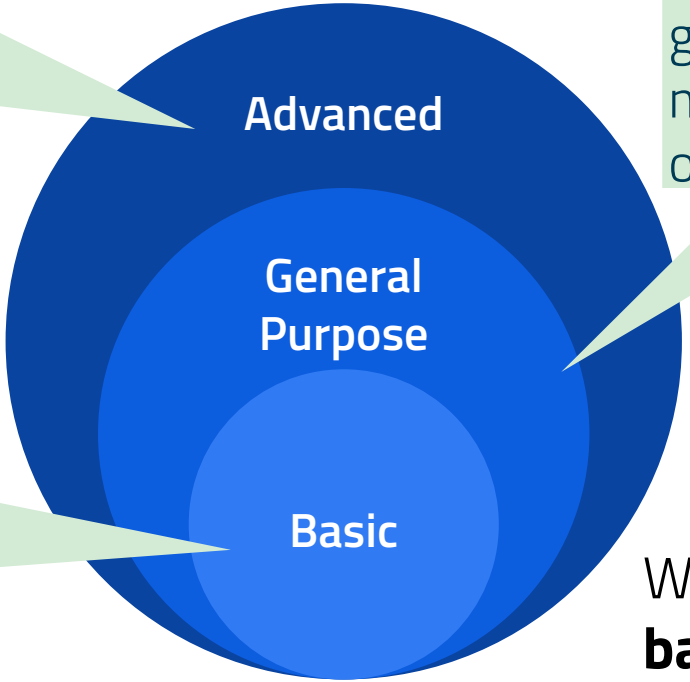
$$\textbf{Period} = \frac{(1+ARR) \times (PSC+1)}{f_{CLK}}$$

- If we want to set large periods the numerator will be very large (f_{CLK} is in the order of 10^6)
- We need to decide if we want to make PSC large and ARR low, vice-versa, or trade-off
- Large ARR are used to have a higher resolution (some timers have 32-bit ARR registers!)
- Large value of prescaler reduces timer's resolution
- However, large prescaler values improve energy efficiency!

Timers categories

General purpose + specific features: for motor control, power conversion, etc

Simplest, 16-bit resolution, can be used as basic time-base generation



Basic + many features: PWM generation, one-pulse mode, input capture, output compare

We will focus on **basic** and **general purpose** timers

Basic Timers

Basic Timers

- Simplest timers available. All STM32 timers implement **at least** the features of basic timers
- A general-purpose timer or an advanced-timer must implement all functionalities of a basic timer!
- They have a 16-bit resolution
- They can be used for time-base generation (to generate periodic update events)
- In HAL all the functions related to programming the “basic” part of a timer are performed through **`HAL_TIM_Base_XXX`** functions

Timers in HAL

```
typedef struct {
  TIM_TypeDef *Instance; /* Timer Register base addr */
  TIM_Base_InitTypeDef Init; /* TIM Time Base params */
  HAL_TIM_ActiveChannel Channel;
  DMA_HandleTypeDef *hdma[7]; /* DMA handlers */
  HAL_LockTypeDef Lock; /* Locking object */
  __IO HAL_TIM_StateTypeDef State; /* timer state  */
} TIM_HandleTypeDef;
```

Timer handle: `TIM_HandleTypeDef`

- `Instance`: pointer to the descriptor we are going to use. For example, `TIM6` (`TIMX` defined `stm32f446xx.h`)
- `Init`: instance of C struct `TIM_Base_InitTypeDef` , which is used to configure the base timer functionalities
- `Channel`: number of active channels for timers that provide one or more input/output channels (not used in the case of basic timers)
- `State`: this is used internally by the HAL to keep track of the timer state.

Configuring a timer: TIM_Base_InitTypeDef

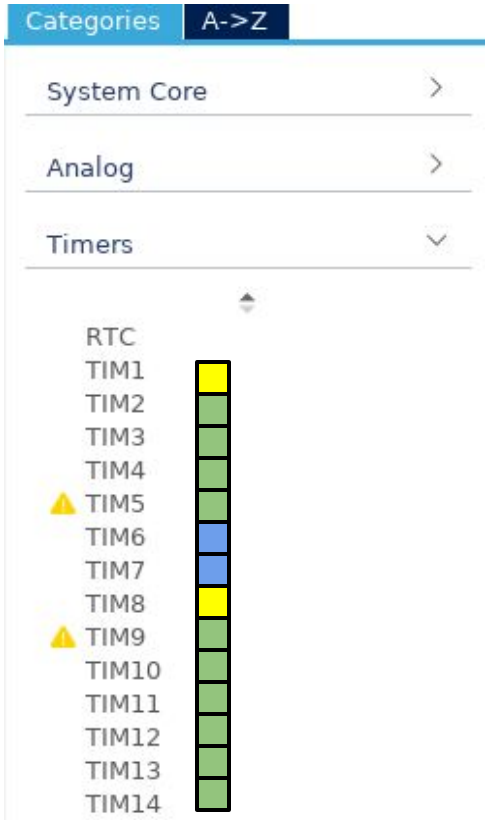
```
typedef struct {
  uint32_t Prescaler; /* Prescaler value (PSC) */
  uint32_t CounterMode;  /*Counter mode (up, down,cent.)*/
  uint32_t Period; /* Value of Auto-reload reg (ARR) */
  uint32_t ClockDivision; /* clock division */
  uint32_t RepetitionCounter; /* Repetition counter val.*/
} TIM_Base_InitTypeDef;
```

- All parameters of a timer can be changed by modifying this structure and then calling the function
  HAL_TIM_Base_Init(TIM_HandleTypeDef *handle);

TIM_Base_InitTypeDef fields

- `Prescaler` : divides the timer clock by a factor ranging from 1 up to 2^16-1 = 65535 (16-bit even though the field is uint32_t!)
- `CounterMode` : defines the counting direction of the timer: `TIM_COUNTERMODE_UP` (default), `TIM_COUNTERMODE_DOWN`, …
- `Period` : maximum value for the timer counter before it restarts counting again (ARR register)
  - 16-bit timers: from 0x1 to 0xFFFF (65535)
  - 32-bit timers: from 0x1 to 0xFFFF FFFF (not for basic timers!)
  - **If Period = 0 => timer does not start!**
- `RepetitionCounter`: can be used to increase period. Update events are generated when the number of times the counter resets equals the value of `RepetitionCounter`

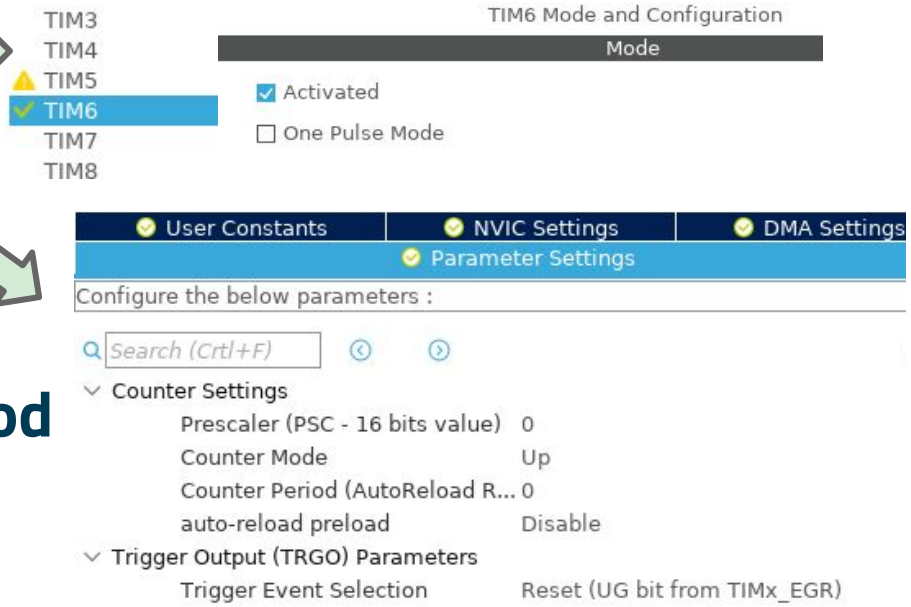
STM32F446XX available Timers



- 14 timers are available on the STM32F446XX
  - Two **basic timers** (blue squares) TIM6, TIM7
  - Two **advanced timers** (yellow squares) TIM1, TIM8
  - The remaining 10 are **general purpose timers** (green squares)

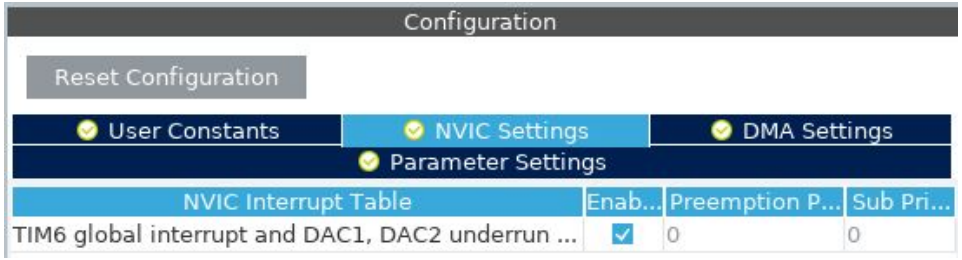
Configuring timers in CubeMX

- First locate the timer you want to enable
- Then click on “activate”
- Once the timer is activated it can be configured using the configuration panel
- To set the period configure **prescaler** and **Counter Period**
- CubeMX reports the resolution of the timer in the “counter period” tooltip



Configure timer to generate interrupts

- In the NVIC settings panel click on “Enable” checkbox to enable the timer to generate interrupts



- Once the code is generated the code to be executed when the timer generates an update event should be implemented in the callback:

  `HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim)` that is called in the ISR `TIM6_IRQHandler(void)` of the corresponding IRQ

Example

```
void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim) {
/* Need to check which timer generated the interrupt! */
    if(htim->Instance == TIM6) {
        HAL_GPIO_TogglePin(LD2_GPIO_Port, LD2_Pin);
    } else if (htim->Instance == TIM7) {
        /* Do something else due to TIM7 interrupt */
    }
}
```

Starting a timer with HAL

- Once the timer has been configured, the timer can be started using:
- `HAL_TIM_Base_Start(TIM_HandleTypeDef *hdl)` if the timer is not configured in interrupt mode (**no interrupts will be generated**!)
- `HAL_TIM_Base_Start_IT(TIM_HandleTypeDef *hdl)` if the timer is configured in interrupt mode
- NOTE: **always** double check that you have called the right function if interrupts are not generated!
- Using a timer in polling mode is quite useless so we will practically always use `HAL_TIM_Base_Start_IT`

Stopping a timer in HAL

- Similarly to starting a timer HAL provides the following functions to stop timers:
  - `HAL_TIM_Base_Stop(TIM_HandleTypeDef *hdl)` for timers in polling mode
  - `HAL_TIM_Base_Stop_IT(TIM_HandleTypeDef *hdl)` for timers in interrupt mode
- Notice that stopped timers can be started again

Configuring basic timer params at runtime

It is possible to configure at runtime the parameters of a timer using some macros:

- To set the prescaler:
  ```
  __HAL_TIM_SET_PRESCALER(TIM_HandleTypeDef *hdl,
  uint32_t prescaler);
  ```
- To set the period (ARR):
  ```
  __HAL_TIM_SET_AUTORELOAD(TIM_HandleTypeDef *hdl,
  uint32_t period);
  ```
- **IMPORTANT TIP:** after using these macros it is advisable to also modify the EGR register of the timer as follows:

  ```
  hdl->Instance->EGR = TIM_EGR_UG;
  ```

Example: setting timer params

```
void set_basic_timer_params(TIM_HandleTypeDef *hdl,
uint32_t prescaler, uint32_t period)
{
    __HAL_TIM_SET_PRESCALER(hdl, prescaler);
    __HAL_TIM_SET_AUTORELOAD(hdl, period);
    hdl->Instance->EGR = TIM_EGR_UG;
}
```

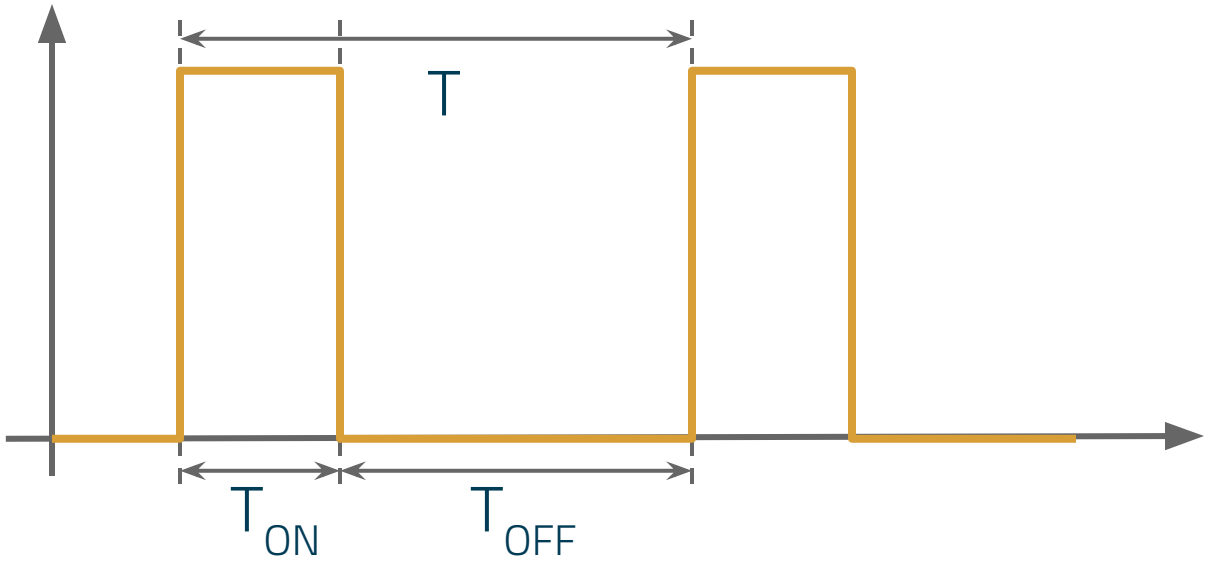
PWM Generation

General purpose timers

- The most common timers provided by STM32 micros
- GP timers are very versatile. They can:
  - be “clocked” by external clocks connected through a micro pin (input capture mode)
  - they can serve as configurable “clocks” for external circuits or be chained with other timers (master/slave timers)
  - they can implement **Pulse Width Modulated** signals routed to specific MCU pins
- Each GP timer can have up to four channels configurable to implement those functions
- We will only study PWM generation

Pulse Width Modulated signals

- PWM signals are square wave signals running at a specific frequency and with a given duty cycle
- Duty cycle = $T_{ON}/T \times 100$

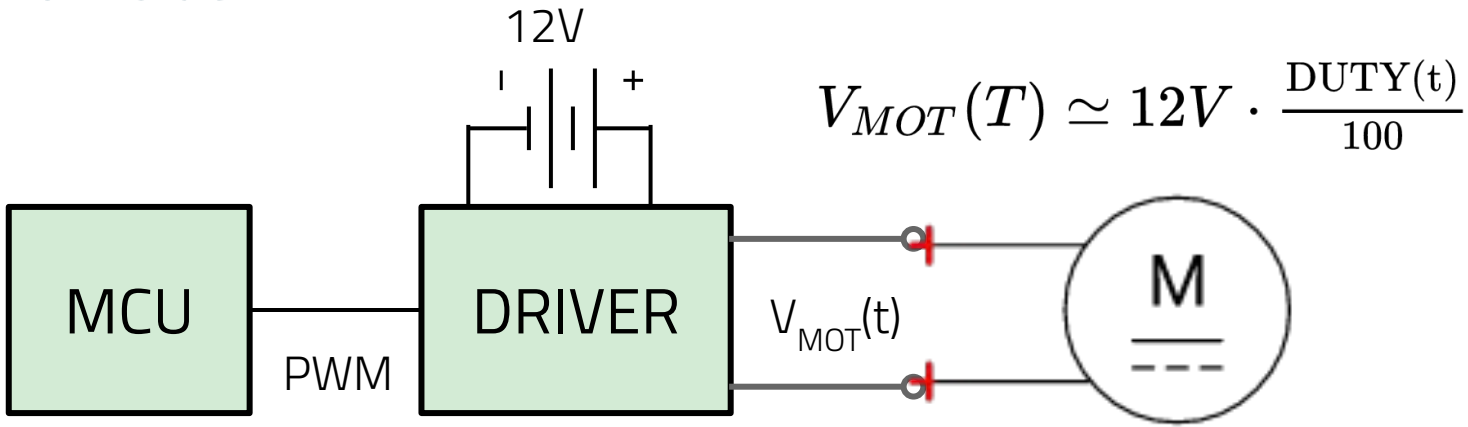


Why PWM signals are interesting

- Suppose we want to actuate the output our controller through a given voltage $V(t_k)$
- This is quite typical in the context of DC motors
- We need to convert a digital value to an analog voltage
- In principle we could use a DAC and then a voltage amplifier. This can be done but is not cheap
- A cheaper way is to "encode" the analog value $V(t_k)$ in a square wave whose **average** is equal to $V(t_k)$
- This can be done easily with a PWM signal that drives a MOSFET connected to a constant voltage supply

Example (very simplified!)

- We want to regulate the speed of a motor
- We can use a “driver” (typically an NMOS H-Bridge) to regulate the voltage and thus the velocity of the motor shaft
- We use the PWM signal to drive a power MOSFET on the low side

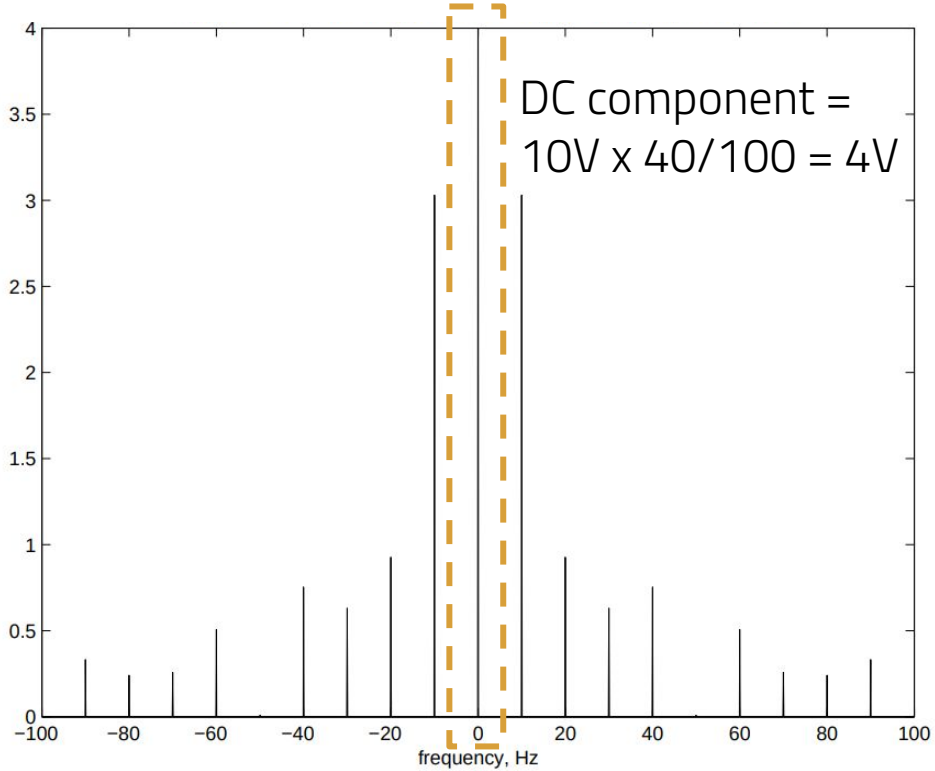


$$V_{MOT}(T) \simeq 12V \cdot \frac{\mathrm{DUTY(t)}}{100}$$

Selection of frequency

- Frequency spectrum of an ideal square wave depends on the frequency of the square wave
- In general, the spectrum is the sampling of the cardinal sine (sinc(2*pi*f))
- The DC component is equal to the average of the signal (i.e., the duty cycle multiplied by the max of the square wave)
- The larger the frequency the more “sparse” the spectrum

Example: 40% duty cycle, freq = 10Hz



- If not filtered, high frequency component might induce resonance in the plant
- Plants are typically low pass filters
- We can tune the PWM frequency so that harmonics are filtered by the plant

PWM generation with STM32 timers

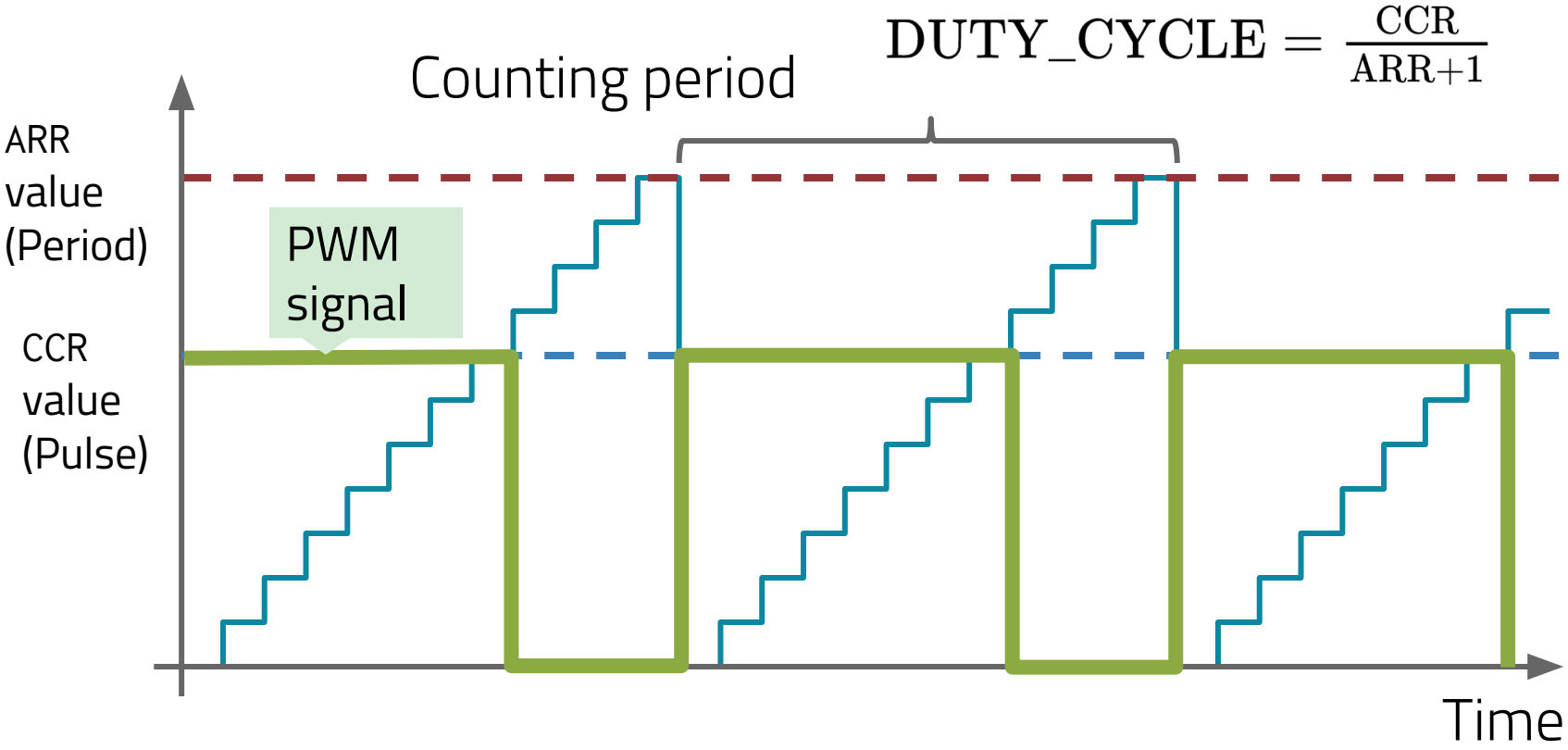
- Timers can be used to generate PWM signals at a given frequency (limited by the clock feeding the timer) and duty cycle
- The resulting square wave oscillates between 0 and VDD (=3.3V)
- The PWM generation exploits a feature of **general purpose timers** (not available for basic timers) called “output compare”
- For each general purpose timer several PWM signals can be generated through “channels”

PWM generation with STM32 timers

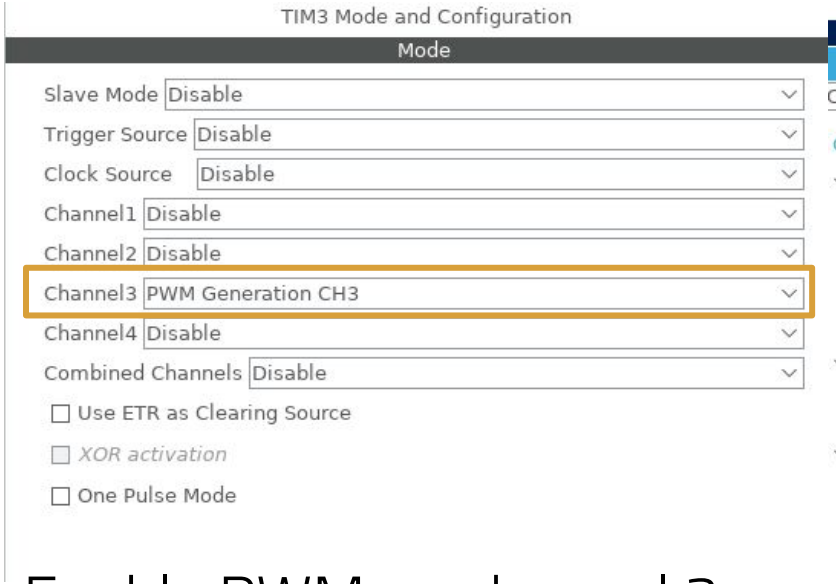
- Each channel of a given timer is associated to a physical pin of the MCU where the PWM signal will be generated
- Timer frequency is selectable as we have seen for basic timers setting prescaler (PSC) and period (ARR)
- Duty cycle can be selected by changing the "pulse" value stored in the CCR register
- Two PWM modes:
  - Mode 1: in upcounting, the channel is active as long as CNT <CCR, else inactive.
  - Mode 2: in upcounting, channel 1 is inactive as long as CNT < CCR, else active

PWM duty cycle

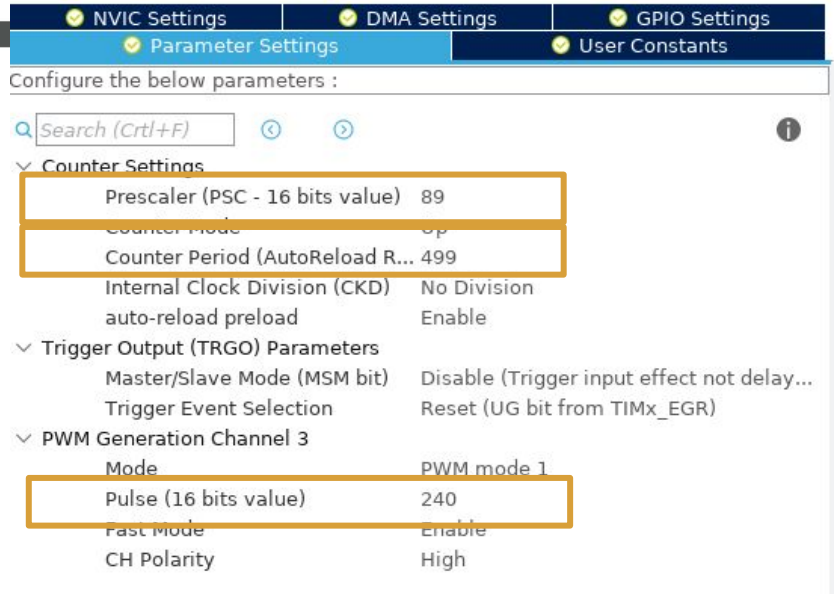
$$\text{DUTY_CYCLE} = \frac{\text{CCR}}{\text{ARR}+1}$$



Configuring PWM in CubeMX



Enable PWM on channel 3



Configure the channel to have a given period and duty cycle

Using PWMs in control loops

- First configure the PWM frequency acting on PSC and ARR (exactly the same formulas used for basic times - slide #11)
- PWM frequency setting depends on the plant we need to control (too low means higher harmonics will pass through the plant and will be “visible” at the system’s output)
- Start the PWM with
  `HAL_StatusTypeDef HAL_TIM_PWM_Start(TIM_HandleTypeDef *htim, uint32_t Channel)`
- **`Channel`** is the PWM channel you want to start (channel 1 to 4). Valid values are `TIM_CHANNEL_1`, … , `TIM_CHANNEL_4`

Using PWMs in control loops

- The controller computes the output that needs to be converted to the duty cycle which belongs to [0,1]
- Say **`duty_cycle`** is the value we want our PWM to produce
- Compute the value of the CCR register to impose such a DC:

```
ccr = (uint16_t)(duty_cycle*(float)(1+ARR))
```

- Assign this value to the CCR register:

```
__HAL_TIM_SET_COMPARE(hdl, TIM_CHANNEL_X, ccr);
```

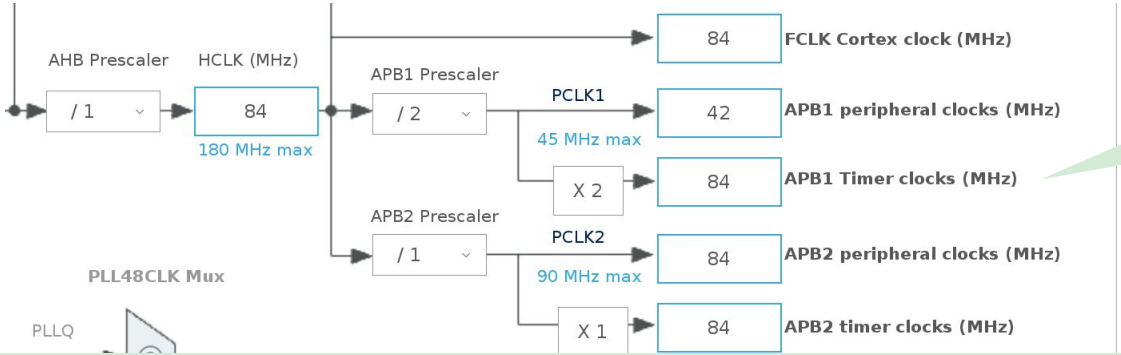
- Call the following:

```
hdl->Instance->EGR = TIM_EGR_UG;
```

to make sure the duty cycle gets updated

Example

- Set timer `TIM2` (GP timer) in PWM mode on channel 2
- We want to set a PWM frequency of f_{PWM}=**1KHz**
- First let's check what is the source frequency of `TIM2`. The manual says `TIM2` is connected to APB1 timer bus (also check the **`#define TIM2_BASE`** in **`stm32f446xx.h`**)
- Locate APB1 timer clocks in clock configuration in CubeMX



Here it is!
84MHz

Example

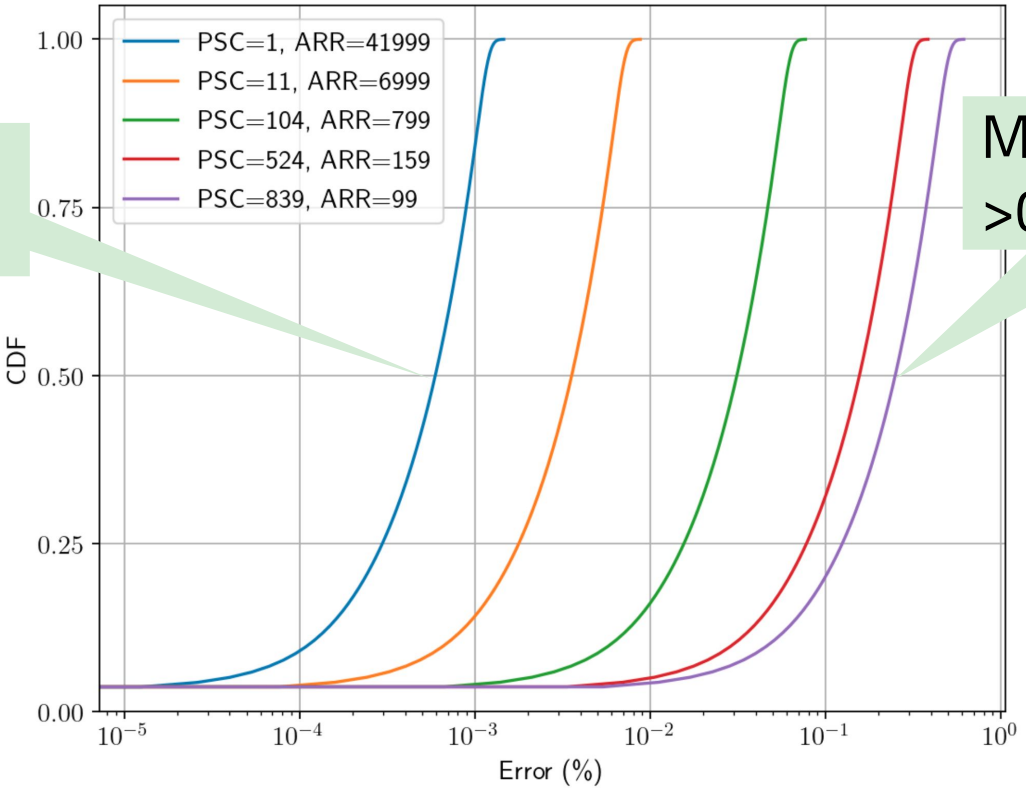
- Set PSC and ARR accordingly: $f_{\mathrm{PWM}} = \dfrac{f_{\mathrm{APB1}}}{(\mathrm{ARR}+1)(\mathrm{PSC}+1)}$
- We readily obtain: $(\mathrm{ARR}+1)(\mathrm{PSC}+1) = \dfrac{f_{\mathrm{APB1}}}{f_{\mathrm{PWM}}}$

$$= \frac{84 \cdot 10^6\,\mathrm{Hz}}{1 \cdot 10^3\,\mathrm{Hz}} = 84000$$

- Select the lowest possible PSC such that ARR can be the highest and (ARR+1)(PSC+1) is as close as possible to 84000 (in this case PSC = 1, ARR = 41999)

Duty cycle precision as **ARR** increases (f_{APB1}=84MHz)

Median error
< 0.001%



Median error
>0.1%

<u>Take home message</u>
keep PSC as low as possible to allow a large ARR

References

"General-purpose timer cookbook for STM32 microcontrollers", ST Application Note AN4776, July 2019, https://bit.ly/2S4kKLU

"STM32 cross-series timer overview", Application Note AN4013, https://bit.ly/3tU7NkY